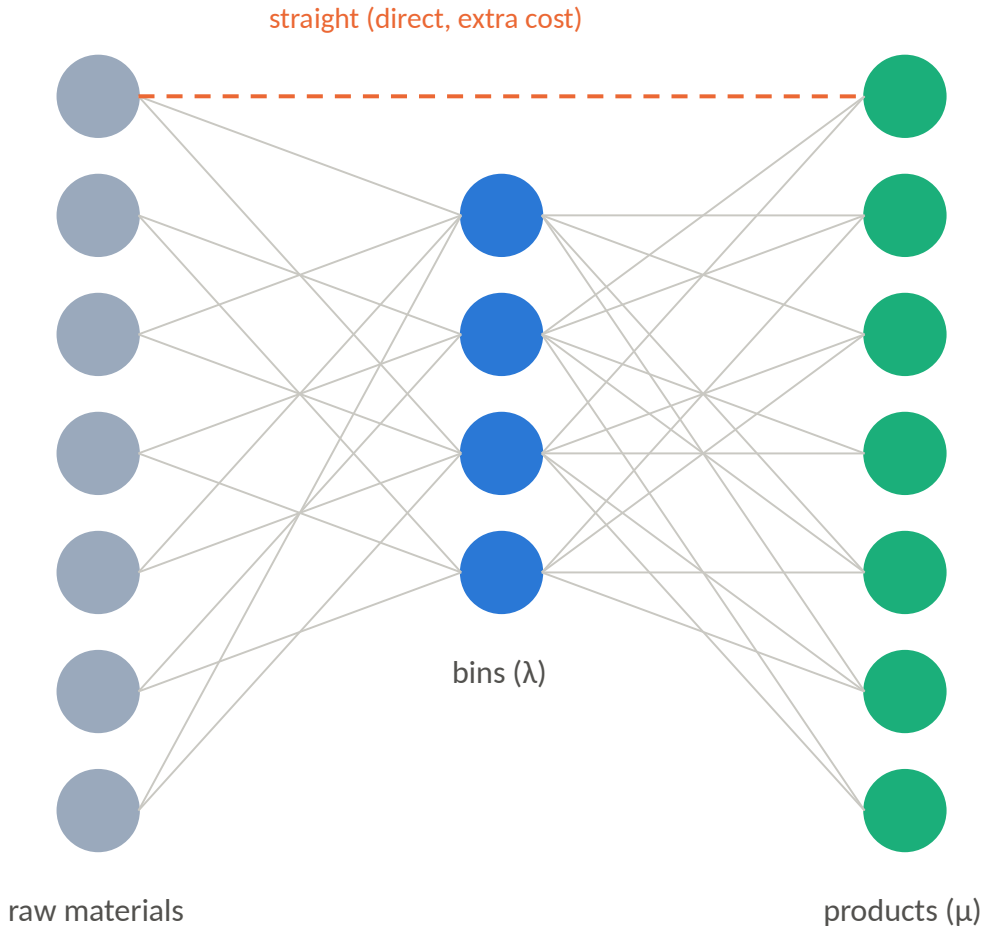
Recipe-Guided Iterated Local Search around SLP for the Fixed-Demand Pooling Problem

IA-2 · Introduction to Optimization Techniques · extending our IA-1 filter-SLP code

Group OPT-09 · Razzan Tambe (15) · Saurabh Tripathi (04) · Vivek Sharma (12)
Dept. of IT, K. J. Somaiya College of Engineering

Shared bins make feed blending nonconvex



- Fixed demands t_p must meet N nutrient specs
- Bin compositions λ are decisions, shared by many products
- Product quality = Σ μ_pm · v_mn → bilinear, nonconvex
- Many local optima even with fixed demands (96 in a 2-nutrient toy example)
- Mills re-solve many times a day → need near-optimal recipes in seconds

Baseline: the paper's method, as implemented in our IA-1 code

qq-formulation (Grothey & McKinnon, 2023)

$$\min \Sigma_p t_p c^D_p$$

$$v_{mn} = \Sigma_i \lambda_{mi} r_{in}$$

$$d_{pn} = \Sigma_i \mu_{pi} r_{in} + \Sigma_m \mu_{pm} v_{mn} \quad \text{(bilinear)}$$

$$c^D_p = \Sigma_i \mu_{pi} c_i + \Sigma_m \mu_{pm} c^M_m \quad \text{(bilinear)}$$

$$\Sigma \lambda = 1,\ \Sigma \mu = 1,\ \underline{d} \le d \le đ,\ \text{bounds}$$

- SLP: linearise, solve LP in an ∞-norm trust region, accept with a filter
- Random multistart: every run independent
- IA-1: qq_model.py + slp.py (Python, HiGHS); af-1 best known 296.44604 reproduced

863×

faster to a good solution than FilterSQP

20×

faster than the global solver Baron

But on the hard instances only

4–20%

of random starts end within 0.2% of the best known cost (af-2, af-3, af-4, af-6, af-6-fs, af-7)

Research gap: random restarts waste most of the run time

Instance	Paper %Good	Paper ETiGood	IA-1 code %Feas	IA-1 code %Good
af-2	19.4%	6.7 s	95%	0%
af-3	20%	6.5 s	100%	0%
af-4	19.8%	8.6 s	60%	25%
af-6-f	4%	112.5 s	65%	5%
af-6-fs	5.2%	125 s	90%	0%
af-7	20.4%	22.5 s	75%	5%

IA-1 code = our IA-1 notebook, 20 runs, seed 12345 (the IA-1 report validated af-1 only: 20/20 Good; af-2, af-6-f, af-7b-fs re-run with the same code and seed). af-6-f = the paper's af-6. ETiGood = 100·Ti / %Good.

What we observed

- Runs land in poor basins; nothing is learned from earlier runs
- Some runs stall in feasibility restoration (af-4, af-7)
- The structure is unused: fix the bins, or the bin shares, and the problem is an LP
- Gap found via: limitation stated by the authors + IA-1 implementation experience + IA-1 results vs the paper

RG-ILS-SLP: same IA-1 model and solver, smarter search

1 New

Diet LP

each product's ideal recipe + lower bound

→

2 Modified

Recipe clustering

weighted k-means on recipes → initial bins

→

3 New

LP repair

product LP / bin LP until the start is feasible

→

4 Retained

Filter-SLP

IA-1 slp.py, unchanged

→

5 New

LP polish

exactly feasible, never worse

6 **Regret-guided iterated local search (new).** From the best solution so far, build 4 candidates: re-seed a bin with the ideal recipe of a product that pays a lot for bin sharing (regret), LP-repair, probe with a 10-iteration SLP; continue the full SLP from the best probe. Runs 1–3 and any run after 6 failures use a fresh recipe start. One pass = one run; 20 runs.

Three exact LPs exploit the bilinear structure

Diet relaxation

$c^*_p = \min \Sigma\, c_i\, w_{pi}$
s.t. specs on $\Sigma\, w_{pi}\, r_{in}$, $\Sigma\, w = 1$
$LB = \Sigma\, t_p\, c^*_p \le f^*$

af-1: LB 295.932 = Baron's 2-hour bound

Product LP (bins fixed)

λ fixed $\Rightarrow$ v, c^M constants
(5c), (5g) become linear in μ
$\rightarrow$ optimal recipes, exact feasibility

used to repair starts and polish SLP end points

Bin LP (bin shares fixed)

μ_{pm} fixed $\Rightarrow$ d linear in (λ, μ^S)
alternate with product LP
$\rightarrow$ cost never increases

classical recursion / ALT idea (Haverly; Audet et al.)

Recipe regret $\text{regret}_p = t_p \cdot \max(0, c^D_p - c^*_p)$. Pick product p $\propto$ regret and a bin m, set $\lambda_m \leftarrow \text{Proj}((1-\alpha)\, q_p + \alpha\, \lambda_m)$, $\alpha \sim U(0, 0.5)$; 4 such candidates are ranked by a 10-iteration SLP probe. The model (5a)–(5h) itself is unchanged.

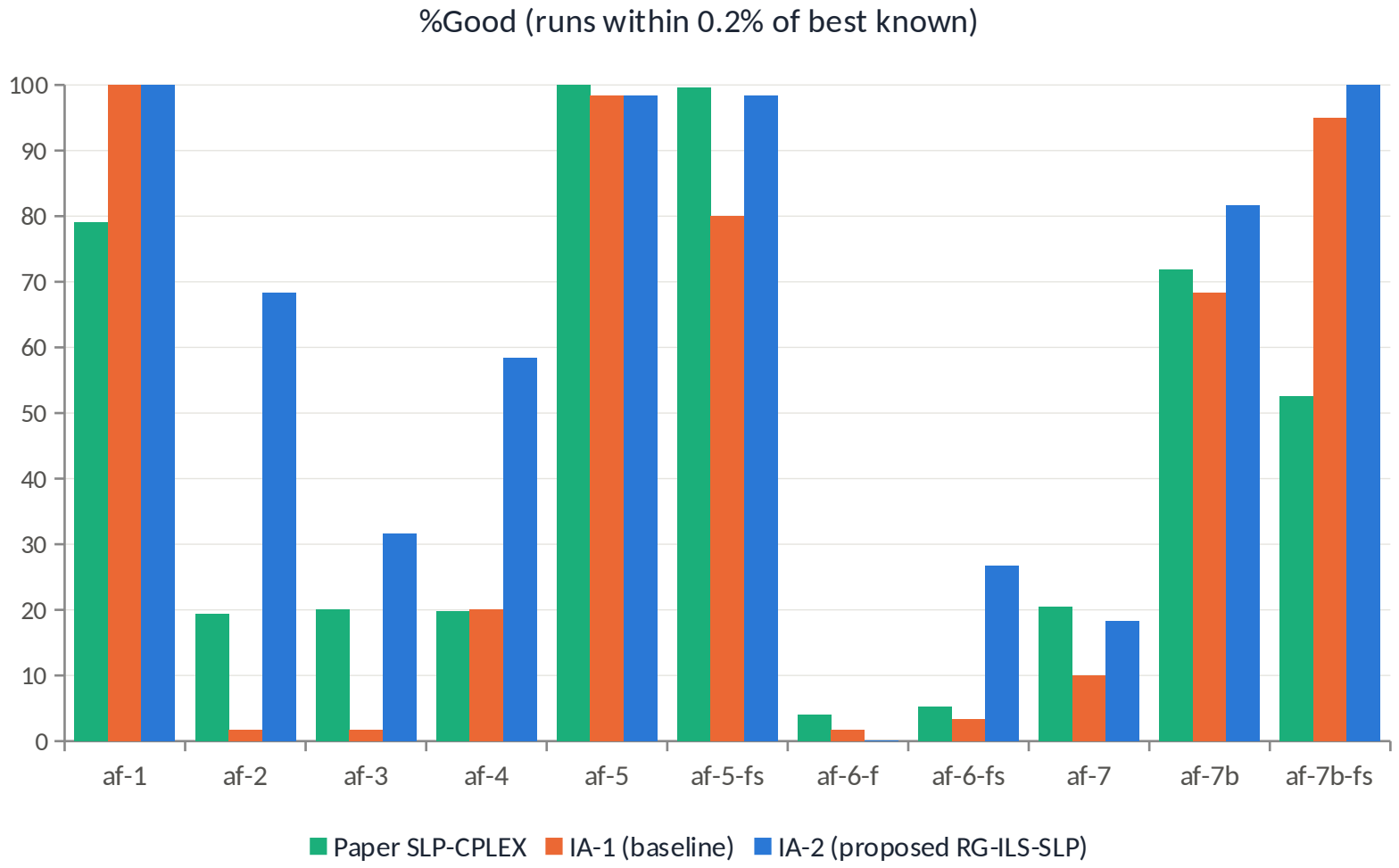
Experimental setup

Item	Details
Instances	all 11: af-1, af-2, af-3, af-4, af-5, af-5-fs, af-6-f (= paper's af-6), af-6-fs, af-7, af-7b, af-7b-fs
Code	IA-1 qq_model.py + slp.py unchanged; new rg_ils.py; run_experiments.py --method
Software / HW	Python 3.13, NumPy, SciPy 1.18 (HiGHS LP); 2-vCPU cloud container, 1 thread per run
Runs	3 seeds (12345 = IA-1 seed, 2024, 7) × 20 runs per instance and method
IA-1 vs IA-2	IA-1 = baseline (our IA-1 code, unchanged; seed 12345 reproduces the IA-1 notebook); IA-2 = proposed RG-ILS-SLP
Metrics	%Feas, %Good (≤ 0.2% of best known), Ti, It, ETiGood, gaps; Fisher and Mann–Whitney tests
Also compared	published SLP-CPLEX (500 runs); 2 ablations (seed 12345)

IA-1 vs IA-2 at a glance

	IA-1 (baseline)	IA-2 (proposed)
Model + SLP solver	qq_model.py + slp.py	same files, unchanged
Starting points	uniform random	recipe clustering (diet LP) + LP repair
Restarts	independent random multistart	regret-guided ILS, 4 SLP-probed moves, fresh starts after 6 failures
SLP end point	returned as is	LP polish (product LP, bin LP)
Lower bound	none	diet LP (af-1 = Baron's bound)
Mean %Feas (11 inst.)	87.9%	96.1%
Mean %Good (11 inst.)	43.6%	62.0%
%Good, 6 hard inst. (paper 14.8%)	6.4%	33.9%
ETiGood, hard inst.	1.0×	5.0× shorter

IA-2 vs IA-1: many more runs reach a Good solution on the hard instances



68%

af-2 (paper 19.4%, IA-1 1.7%)

58%

af-4 (paper 19.8%, IA-1 20.0%)

0%

af-6-f (paper 4%, IA-1 1.7%) — not solved

IA-2 vs IA-1: more feasible runs, faster to a good solution

88% → 96%

mean %Feas, IA-1 → IA-2

44% → 62%

mean %Good, IA-1 → IA-2

5.0×

shorter ETiGood on the hard instances (geo-mean, af-6-f excluded)

Instance	Paper ETiGood	IA-1 ETiGood	IA-2 ETiGood
af-2	6.7 s	60.1 s	**3.1 s**
af-3	6.5 s	60.9 s	**7.0 s**
af-4	8.6 s	16.9 s	**10.7 s**
af-6-f	112.5 s	369.6 s	**no Good run**
af-6-fs	125.0 s	368.3 s	**50.9 s**
af-7	22.5 s	72.6 s	**43.4 s**

Reading the times

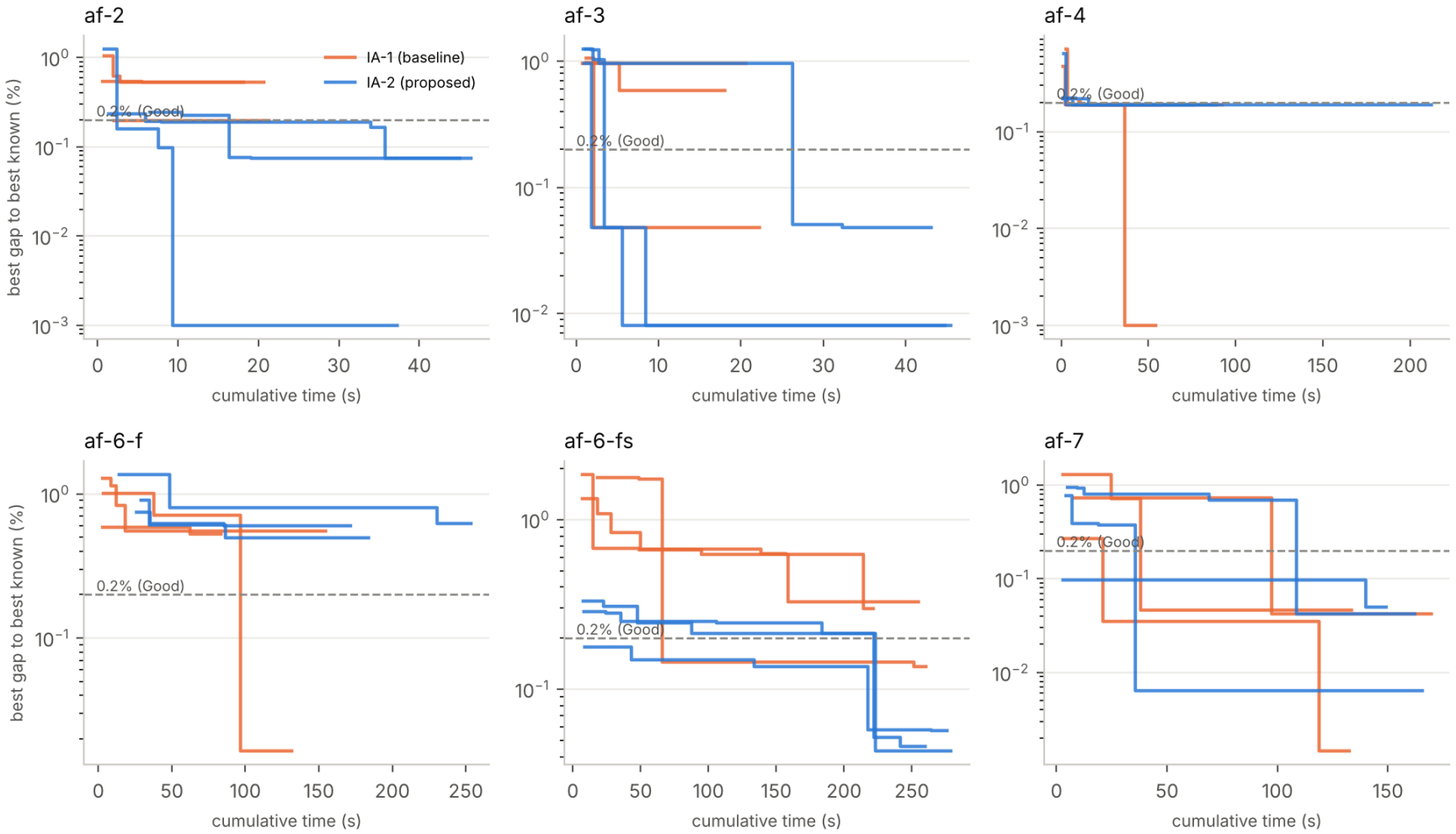
- An IA-2 run costs more (repair, 4 probes, polish) but far fewer runs are wasted
- Easy instances (af-1, af-5, af-5-fs, af-7b-fs): overhead, ETiGood up to ~2× worse
- Paper times: CPLEX in C, 500 runs; ours: HiGHS from Python

The gains are statistically significant

Instance	Good: IA-1 → IA-2	p (Fisher)	Feasible: IA-1 → IA-2	p (Fisher)	p (Mann–Whitney, gaps)
af-1	60/60 → 60/60	1.0000	60/60 → 60/60	1.0000	0.0634
af-2	1/60 → 41/60	< 0.0001	59/60 → 58/60	0.8782	< 0.0001
af-3	1/60 → 19/60	< 0.0001	58/60 → 60/60	0.2479	< 0.0001
af-4	12/60 → 35/60	< 0.0001	41/60 → 52/60	0.0138	< 0.0001
af-5	59/60 → 59/60	0.7521	59/60 → 60/60	0.5000	0.0004
af-5-fs	48/60 → 59/60	0.0010	48/60 → 60/60	0.0001	0.0002
af-6-f	1/60 → 0/60	1.0000	44/60 → 44/60	0.5817	0.7554
af-6-fs	2/60 → 16/60	0.0003	54/60 → 60/60	0.0137	< 0.0001
af-7	6/60 → 11/60	0.1475	42/60 → 60/60	< 0.0001	0.0020
af-7b	41/60 → 49/60	0.0696	57/60 → 60/60	0.1218	0.0023
af-7b-fs	57/60 → 60/60	0.1218	58/60 → 60/60	0.2479	0.0003

One-sided tests, IA-2 better than IA-1, 60 vs 60 runs per instance (3 seeds × 20). Shaded: $p < 0.05$. ILS runs within a seed are dependent, so the per-seed results in the paper (Table A1) are the conservative check.

The guided search keeps the incumbent below the 0.2% line (except af-6-f)



IA-2 best found: af-2 = best known (IA-1 +0.20%), af-3 +0.008% (IA-1 +0.048%), af-6-fs +0.04% (IA-1 +0.14%). On af-4 (+0.19%) and af-6-f (+0.50%) an IA-1 run was better: more reliable, not always better in the single best run.

Best gap to the best-known cost vs cumulative time (log scale), one line per seed; blue = IA-2, orange = IA-1; dashed = Good threshold.

Ablation: the components complement each other

Instance	IA-1 (baseline)	A: random + polish	B: recipe start, no ILS	IA-2 (full)
af-1	100% / 100%	100% / 100%	100% / 90%	**100% / 100%**
af-2	95% / 0%	100% / 0%	100% / 20%	**95% / 65%**
af-3	100% / 0%	100% / 0%	95% / 5%	**100% / 15%**
af-4	60% / 25%	70% / 25%	100% / 25%	**80% / 60%**
af-5	100% / 100%	100% / 100%	100% / 100%	**100% / 100%**
af-5-fs	90% / 90%	100% / 100%	100% / 95%	**100% / 100%**
af-6-f	65% / 5%	70% / 5%	60% / 15%	**80% / 0%**
af-6-fs	90% / 0%	100% / 0%	100% / 20%	**100% / 5%**
af-7	75% / 5%	85% / 5%	95% / 20%	**100% / 20%**
af-7b	95% / 70%	100% / 75%	100% / 90%	**100% / 85%**
af-7b-fs	100% / 100%	100% / 95%	100% / 95%	**100% / 100%**

Polish alone mainly fixes feasibility (af-4 60→70%, af-6-fs 90→100%).

Recipe starts alone are best on af-6-f, af-6-fs and af-7b, but weak on af-2, af-3 and af-4.

The probed ILS adds the big gains on af-2 (65%), af-3 (15%) and af-4 (60%), but over-refines on af-6-f (0%).

Diversity (fresh starts) + depth (ILS) together.

Cells show %Feas / %Good over 20 runs, seed 12345.

Discussion and limitations

Gap addressed

- %Good up on 5 of 6 hard instances (af-7 seed 2024: tie)
- 96% of IA-2 runs feasible (IA-1 88%)
- Significant: Fisher $p < 0.01$ on 5 instances
- IA-1 model and SLP unchanged

Trade-offs

- Extra LPs and probes cost time per run
- Easy instances get slower (ETiGood up to ~2×)
- ILS runs within a seed are dependent → 3 seeds

Limitations

- af-6-f not solved (IA-2 0% vs paper 4.0%, IA-1 1.7%)
- af-7: 18.3% vs paper 20.4%
- af-4 single best run: IA-1 found the best known value
- 3×20 runs vs 500 in the paper; other hardware/LP solver
- Diet bound tight only on af-1

Structure-aware restarts make our IA-1 SLP reliable on hard pooling instances

Mean %Feas 87.9% → 96.1%, %Good 43.6% → 62.0%; hard instances 6.4% → 33.9% (paper 14.8%), ETiGood 5.0× shorter

Future: more seeds / full benchmark, adaptive choice of moves, parallel probes, stronger bounds, capacity limits

References

[1] A. Grothey, K. McKinnon, Annals of Operations Research 322 (2023) 691–711.

[2] M. Terzi et al., Computers & Operations Research 163 (2024) 106507.

[3] S. Sass et al., European Journal of Operational Research 316 (2024) 36–45.

[4] P. E. Gill, M. Zhang, Computational Optimization and Applications 88 (2024) 37–70.

[5] M. Muehlebach, M. I. Jordan, Mathematical Programming 215 (2026) 407–452.

[6] D. di Serafino et al., Mathematical Programming 205 (2024) 107–134.

[7] C. Audet et al., Management Science 50 (2004) 761–776. [8] C. A. Haverly, ACM SIGMAP Bulletin 25 (1978).

[9] A. Grothey, K. McKinnon, Pooling problem test data (2020). [10] R. Fletcher, S. Leyffer, Math. Programming 91 (2002).

[11] H. R. Lourenço et al., Iterated local search, Handbook of Metaheuristics (2019). [12] D. Arthur, S. Vassilvitskii, k-means++, SODA (2007).

[13] Group OPT-09, IA-1 report: SLP for nonlinear pooling optimization problems (2026).